

智能交互机器人实验

刘咏彬

liuyb@bupt.edu.cn

本环节实验目的

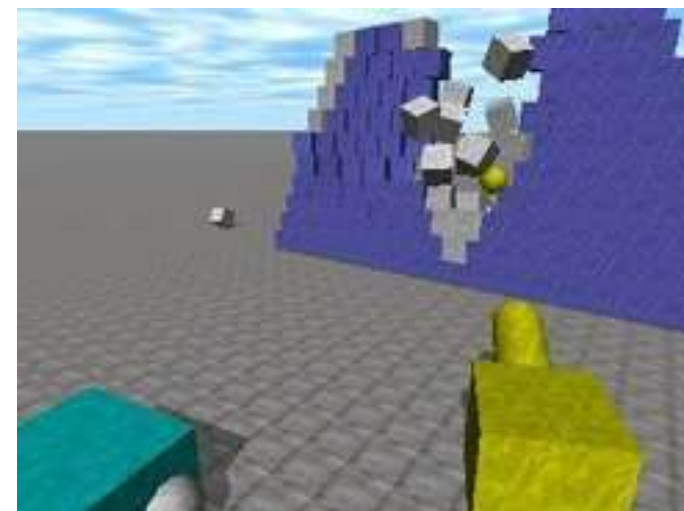
- 掌握机器人整体结构
 - 各类型传感器
 - 输出控制
- 机器人智能决策：寻路避障
- 仿真环境搭建+机器人仿真实验

任务一

- 使用WebotsCoppeliaSim搭建博物馆/展览馆场景，藏品/展品数量4张以上，实物展品2个以上，障碍物若干。地图细节不限。
 - 机器人类型选择，完成同等功能的情况下，分值折扣不同。
 - Nao（标准平台）或 Roban机器人
 - 其他仿人机器人、Aibo等四足机器人（+5创新分）
 - 自建小车、epuck等轮式机器人（-10）
 - 设计机器人导航软件，能够通过简单键盘输入，如“1、2、3、4、5、6 …”即引发机器人在展位之间移动。（70分）
 - 机器人姿态、路径规划。
 - 实现机器人在场馆内的避障。（10分）
 - 实现机器人手指向展品（如果选取仿人机器人等有手臂的机器人）（10分）
 - 实现难度、精细程度、创新度、实验报告质量、视频质量等。（10分）
-
- 为鼓励同学们交流沟通。在教学云平台讨论区积极提问，或给出有效解答的同学。适当增加“参与分”。
 - 注意代码的独创性，可以讨论，不要合作。

机器人仿真环境

- 仿真环境包括物理引擎、三维实体建模、渲染功能
- 使用仿真环境设计、调试机器人：
 - 廉价，不受硬件平台限制
 - 重复多次实验情况下，可以防止实体机器人损耗
 - 易于复现，构建统一评测标准平台
- 常见的机器人仿真环境：
 - Gazebo，是ROS（Robot Operating System 机器人操作系统）默认的仿真软件
 - PyBullet，和Python紧密结合，目前在增强学习 (RL) 中广泛应用。
 - QiBullet，基于PyBullet搭建的NAO机器人仿真环境
 - MATLAB，以研究机械手和路径规划居多
 - V-REP（现更名为CoppeliaSim），教育版免费，包含NAO机器人模型
 - Webots，近几年刚由商业授权转为开源免费，包含NAO机器人模型



Open Dynamic Engine 物理引擎

CoppeliaSim仿真环境

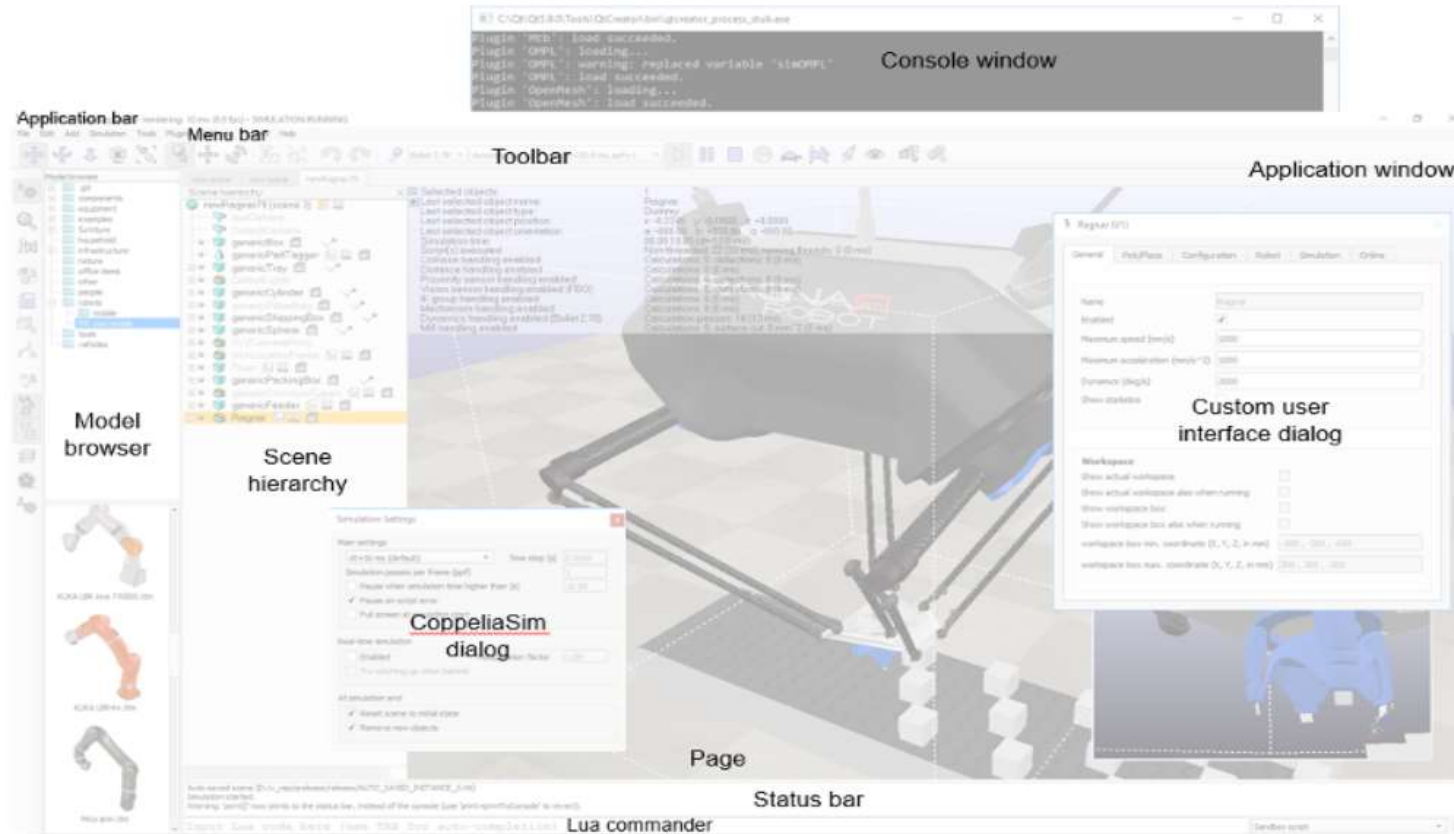
具有**集成开发环境**的机器人模拟器CoppeliaSim基于**分布式**控制架构：每个**对象/模型**都可以通过**嵌入式脚本**，**插件**，**ROS / ROS2节点**，**远程API客户端**或自定义解决方案进行**单独控制**。这使得CoppeliaSim非常通用，是多机器人应用的理想选择。

CoppeliaSim可以用作**独立的应用程序**，也可以方便地嵌入到主客户端应用程序中：其占用空间小，API精心设计，使CoppeliaSim成为嵌入到更高级别的应用程序中的理想候选者。Lua或Python脚本解释器使CoppeliaSim成为一个极其通用的应用程序，让用户可以自由地组合低/高级功能以获得新的高级功能。

CoppeliaSim界面

- 1. a console window:** 在 Windows 下，当 CoppeliaSim 应用程序启动时，会创建一个控制台窗口，但会再次直接隐藏。控制台或终端窗口显示加载了哪些插件及其初始化过程是否成功。**控制台窗口不是交互式的**，仅用于输出信息。可以使用 `print` 命令（从脚本中）或插件中的 `Cprintf` 或 `std::cout` 命令直接将信息输出到控制台窗口。
- 2. an application window:** 应用程序窗口是应用程序的主窗口。它用于显示，编辑，模拟和与场景交互。鼠标左键和右键、鼠标滚轮以及键盘在应用程序窗口中激活时具有特定功能。在应用程序窗口中，输入设备（鼠标和键盘）的功能可能因上下文或激活位置而异。
- 3. several dialogs:** 在应用程序窗口旁边，用户还可以通过**调整对话框设置或参数**来编辑场景并与之交互。每个对话框对一组相关函数或应用于同一目标对象的函数进行分组。

CoppeliaSim界面



CoppeliaSim 应用程序的典型视图

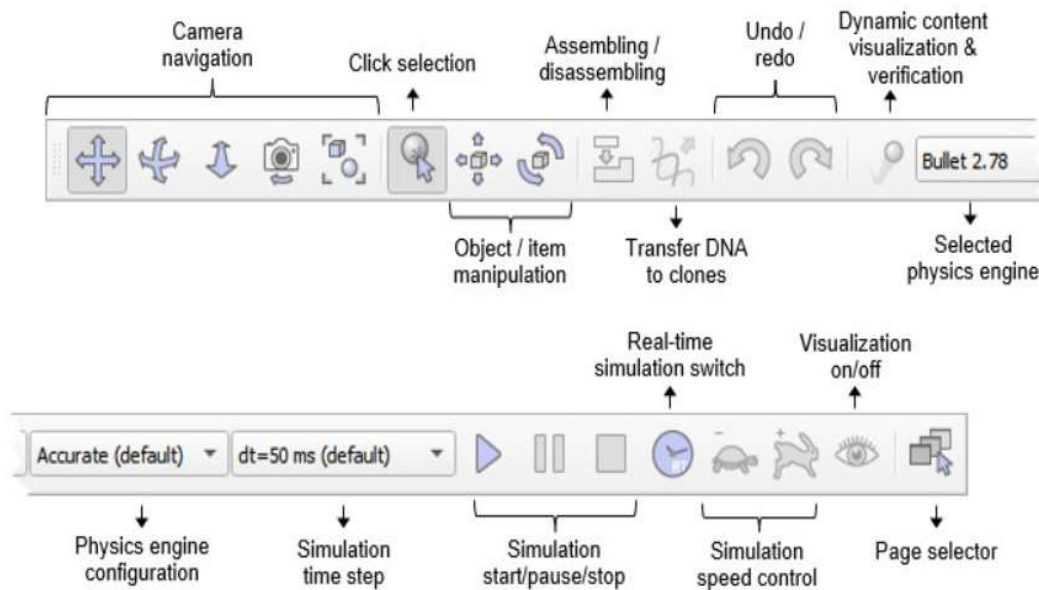
CoppeliaSim界面

应用程序窗口元素

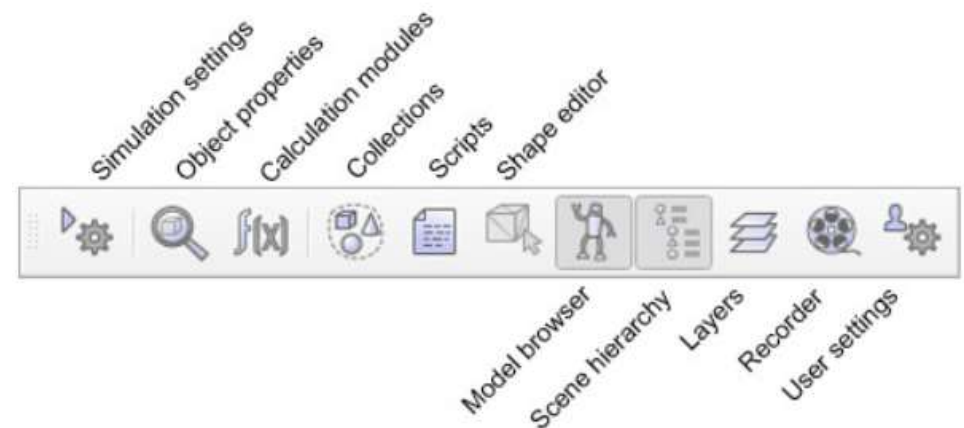
- 1. application bar:** 应用程序栏指示 CoppeliaSim 副本的**许可证类型**、当前正在显示的场景的**文件名**、一个渲染通道所用的**时间**以及模拟器的当前**状态**。应用程序栏以及应用程序窗口中的任何表面也可用于将 CoppeliaSim 相关文件拖放到场景中。支持的文件包括 “*.ttt” 文件和 “*.ttm” 文件。
- 2. menu bar:** 菜单栏允许访问模拟器的几乎所有功能。多数情况下，菜单栏中的项目会**激活对话框**。**菜单栏内容**与上下文相关。菜单栏中的多数功能也可以通过弹出菜单、双击场景层次结构视图中的图标或单击工具栏按钮来访问。
- 3. toolbars:** 工具栏提供经常访问的功能，工具栏 1 中的某些函数和工具栏 2 中的所有功能也可通过菜单栏或弹出菜单访问。工具栏都可以**停靠**和取消停靠，但停靠仅适用于其各自的初始位置。

CoppeliaSim界面

工具栏按钮的功能



[工具栏 1]

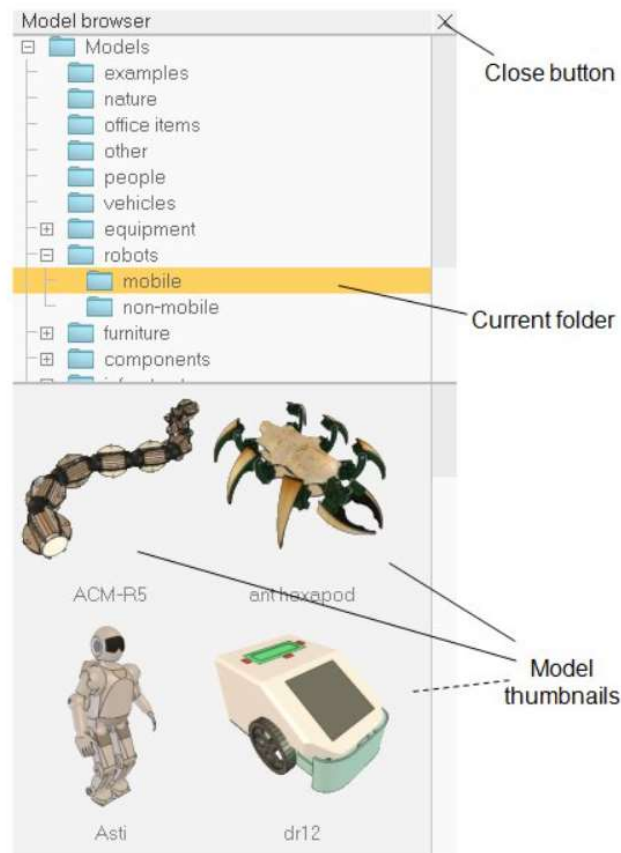


[工具栏 2]

CoppeliaSim界面

应用程序窗口元素

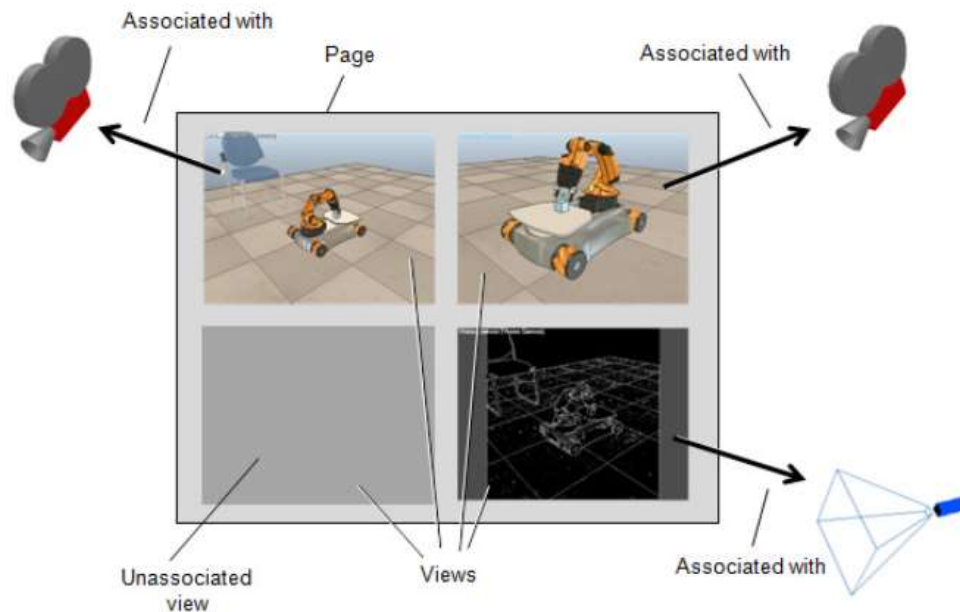
model browser: 模型浏览器默认可见，但可以使用其相应的工具栏按钮进行切换。它的上半部分显示 CoppeliaSim 模型文件夹结构，下半部分显示所选文件夹中包含的模型缩略图。**缩略图**可以拖放到场景中以自动加载相关模型。如果放置区域不受支持或不合适，则捕获的缩略图将显示为深色。



[模型浏览器]

CoppeliaSim界面

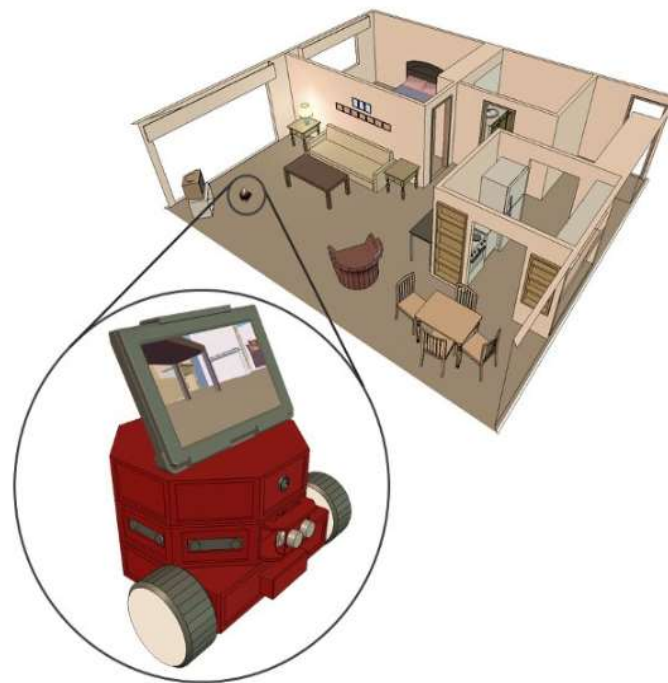
pages and views: CoppeliaSim 中的页面是场景的主要查看表面。它不是直接的视图，但可以根据需要包含一个、两个或任意数量的视图。视图是用于显示特定对象的图像内容，该对象必须是可查看的对象。例如，如果视图与相机对象相关联，则它可以显示相机看到的内容。下图说明了页面、视图和可查看对象的关系如图



[页面、视图和可查看对象关系]

场景与模型

scenes and models是CoppeliaSim的主要模拟元素。模型是场景的**子元素**，明确标记为模型。一个场景可以包含任意数量的**模型**。下图说明了场景-模型关系：**场景和模型**都可能包含以下一个或多个元素：场景对象、子脚本，除了上述元素外，场景还将**包含**以下元素：环境、主脚本、页面和视图，场景保存在“*.ttx”文件中，模型保存在“*.ttm”文件中。这两种文件类型都可以通过从**资源管理器窗口**中拖放到CoppeliaSim 应用程序中来打开。它们也可以通过双击打开。



[场景和模型]

场景与模型

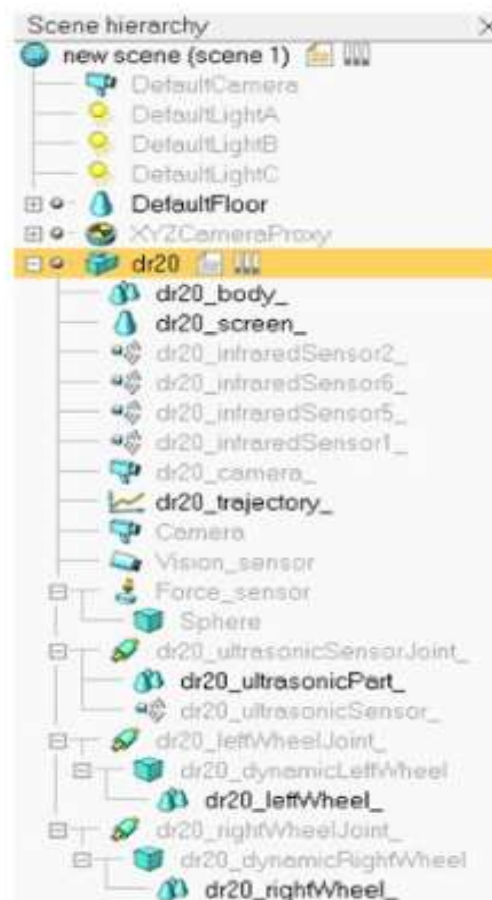
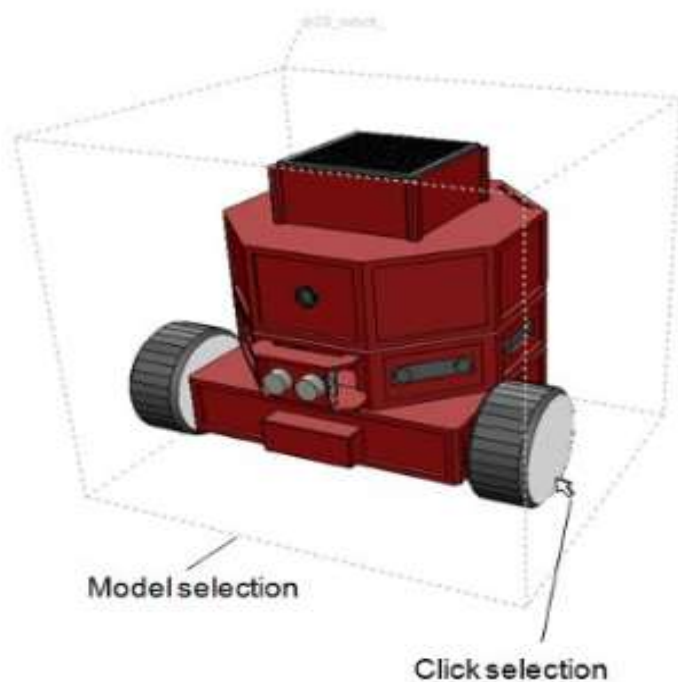
模型是场景的**子元素**。模型本身不能存在，除非在文件（“*.ttm”文件类型）中，也不能单独模拟。模型必须包含在场景中才能运行。

模型由一系列基于同一层次**结构树**构建的场景对象定义，其中树的基础必须是标记为对象是模型基础的对象。

模型在以下步骤中定义：

1. 将**逻辑**上属于模型的所有对象附着到基对象，以便基对象是模型树的基对象。
2. 选中对象是对象**公共属性**中的模型基项。单击“编辑模型属性”，您可以定义特殊的覆盖属性。这允许快速禁用模型中定义的所有对象的某些属性。
3. 对于模型中的所有对象（基本对象除外），选中“对象通用属性”中的“选择模型的基-项”。将**保护您的模型**：您将无法直接选择模型中的单个对象，并且您几乎可以作为单个对象来操作模型。
4. 对于通常**不可见**的所有对象，请选中“不显示为模型内部选择”项。这将使模型边界框以正确的大小显示在模型周围。
5. 在无法在场景中再选择在模型基础上构建的单个对象（选择它们将改为选择模型的基础），但是仍然可以通过在选择期间按住 ctrl 和 shift 键或在场景层次结构中选择它们来**单独**选择它们。除此之外，当选择基本对象时，将显示一个包含整个模型的点状边界框，如下图所示：

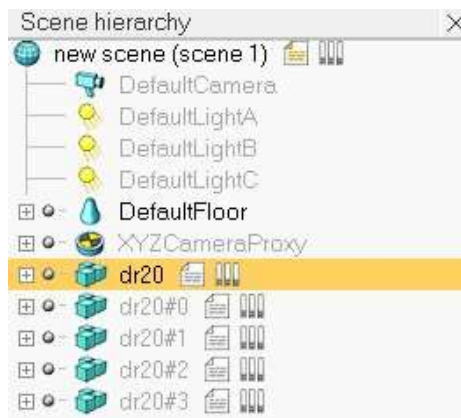
场景与模型



[场景视图和相应的场景层次结构视图。所选虚拟人被标记为对象是模型基础]

场景与模型

双击**模型标记**将打开模型对话框，可在其中调整模型属性。在编辑模型后折叠模型的层次结构也是一种很好的做法，以便轻松识别逻辑分组元素/模型的数量：



[显示 5 个折叠模型的场景层次结构视图]

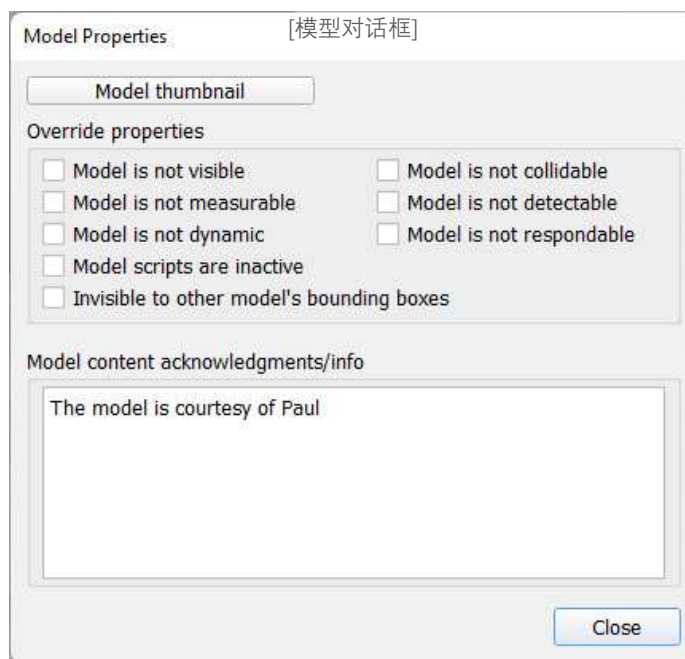
当子脚本以编程方式访问对象时，将多个对象分组为模型也很重要，在CoppeliaSim中，对象/模型可以随时**复制**，也可以在**模拟**期间复制。

复制和粘贴模型的行为与保存模型，然后加载模型（但是使用内存缓冲区而不是磁盘空间）**完全相同**。模型可以像任何其他对象一样从一个场景**复制**到另一个场景。模型文件（“*.ttm”文件）还支持资源管理器窗口和应用程序窗口之间的**拖放**操作。模型文件也可以双击，在这种情况下，它们将启动 CoppeliaSim 应用程序并加载到默认场景中。

场景与模型

model dialog可以在模型对话框中单独调整模型的属性。双击场景层次结构中的模型图标即可打开它：

[场景层次结构中的模型图标]

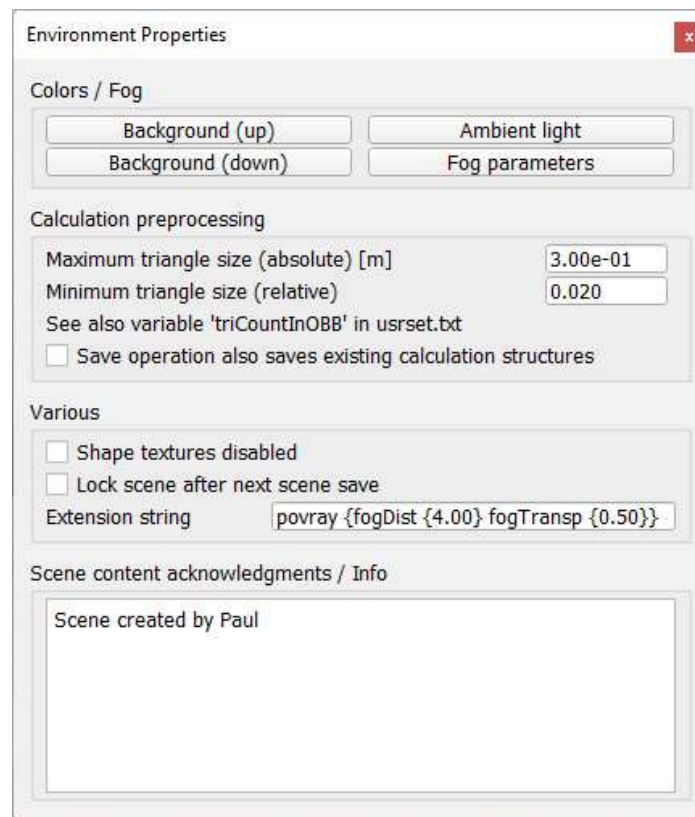


场景与模型

1. **model thumbnail:** 保存模型时，会弹出一个对话框，要求提供模型**缩略图**（将显示在模型浏览器中）。但是，如果希望以**不同的配置**保存模型的缩略图（例如，希望以直线配置保存蛇形机器人的模型，但希望缩略图以弯曲配置可视化蛇形机器人），那么可以在此处指定缩略图。
2. **override properties:** 在这里可以禁用（覆盖）整个模型的**特定属性**（即模型层次结构树中的所有对象）。例如，这便于快速禁用需要太多计算时间的模型。可参阅有关可碰撞对象、可测量对象和可检测对象的部分，以及sim.setModelProperty API 函数。
3. **model content acknowledgments/info:** 与模型相关的信息。**确认模型或导入网格**的原始作者始终是一种很好的做法。打开包含确认信息的模型时，它将自动显示该信息。

环境

environment dialog可以使用 [菜单栏 --> 工具 --> 环境] 或双击场景层次结构中的图标访问环境对话框



[环境对话框]

环境

background (up/down) : 允许调整场景的**背景颜色**。向上分量对应于屏幕的上部，向下分量对应于屏幕的下部。背景色仅在禁用雾功能时可见。

ambient light: 允许调整场景的**环境光**。环境光可以看作是场景的最小光，它从各个方向以完全相同的方式照亮物体。

fog parameters: 允许调整各种**雾参数**。

maximum triangle size: 它影响CoppeliaSim**计算速度**。两个实体之间执行最小距离计算时，如果两个实体都由大小相似的三角形组成，则执行速度通常会更快。

minimum triangle size: 与上一项类似，但此项有助于避免创建可能需要很长时间的太大计算结构。此值将最小三角形大小设置为**相对值**（相对于给定对象的最大尺寸）。

save operation also saves existing calculation structures: 对于**距离计算**、**碰撞检测**等，在模拟开始时（预处理）或第一次涉及此类计算时计算数据结构，以加快计算速度。该数据结构的计算可能非常耗时，因此用户可以选择将其与场景或模型一起保存。

shape textures disabled: 如果选中，则将禁用应用于形状的所有**纹理**。

lock scene after next scene save: 如果要锁定场景不对版本/修改、脚本内容查看和资源导出进行**锁定**，选择此项。完成下一次场景保存操作后，场景将被锁定。

extension string: 描述其他环境属性的字符串，主要由**扩展插件**使用

scene content acknowledgements/info: 与场景相关的信息

功能性

CoppeliaSim提供了由CoppeliaSim本身启用的强大功能，通过插件，附加组件，附加程序或通过扩展模块启用。下面我们概述一下CoppeliaSim的**主要功能**：

- 几何计算
- 运动学
- 动力学
- 数据可视化/输出
- 数据操作/转换
- 消息传递/接口/连接
- 路径/轨迹
- 路径规划
- 合成视觉
- 自定义用户界面
- 导入/导出
- 命令/设置
- 杂项

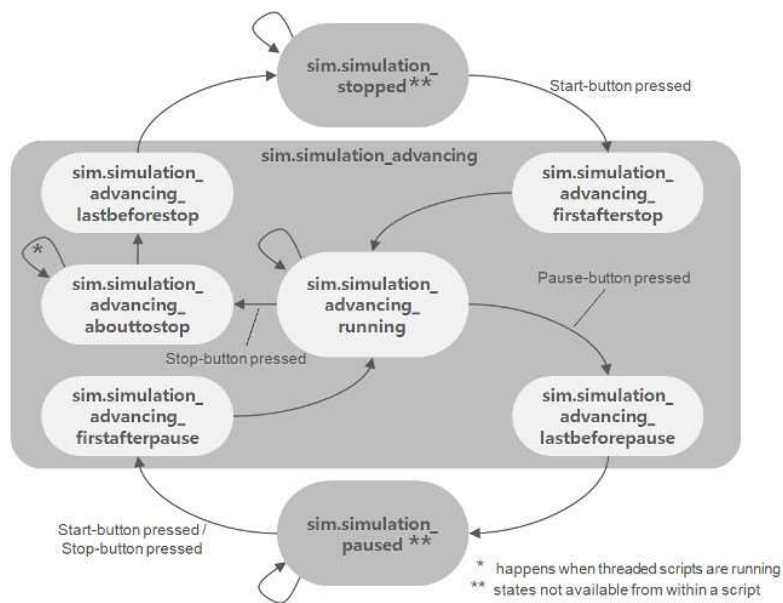
模拟

CoppeliaSim中的模拟可以通过[菜单栏-->模拟-->开始/暂停/停止模拟]或通过相关的工具栏按钮启动，暂停和停止：



[模拟开始/暂停/停止工具栏按钮]

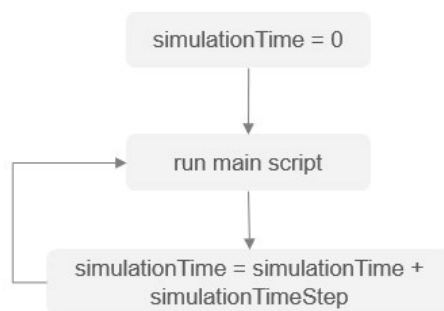
在内部，模拟器将使用其他中间状态，以便正确地通知脚本或程序接下来会发生什么。以下状态图说明了模拟器的内部状态：



[模拟状态图]

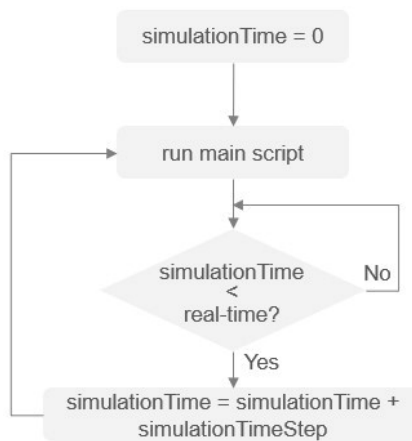
模拟

simulation loop 模拟器通过以恒定时间步长推进模拟时间来运行。下图说明了主要的模拟循环：



[主模拟循环]

通过尝试保持仿真时间与实时同步来支持实时仿真：



[实时模拟循环]

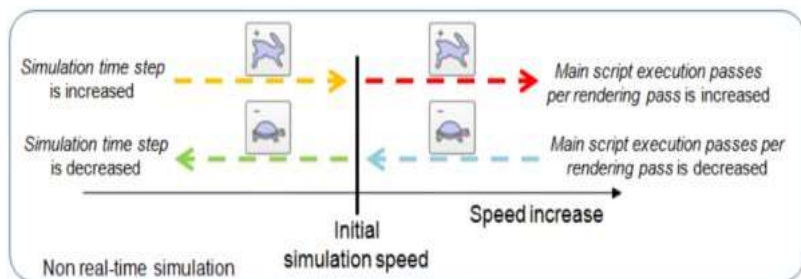
模拟

simulation speed在非实时仿真中，仿真速度（即感知速度）主要取决于**两个因素**：仿真时间步长和一个渲染通道的**仿真通道数**（有关更多详细信息，请参阅模拟对话框）。在实时仿真的情况下，仿真速度主要取决于实时乘法系数，但也在一定程度上取决于仿真时间步长（由于计算机的计算能力有限，太小的仿真时间步长可能与仿真的实时性不兼容）。在仿真过程中，可以使用以下工具栏按钮调整仿真速度：

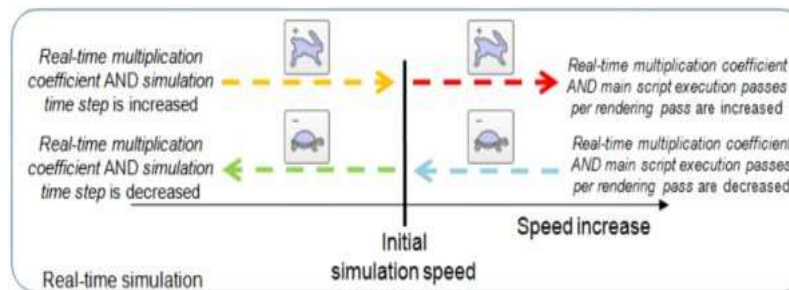


[模拟速度调整工具栏按钮]

以某种方式调整仿真速度，以便初始仿真时间步长永远不会增加（因为例如，这可能会破坏机制）。下图说明了模拟速度调整机制：



[非实时仿真的仿真速度调整机制]



[用于实时模拟的模拟速度调整机制]

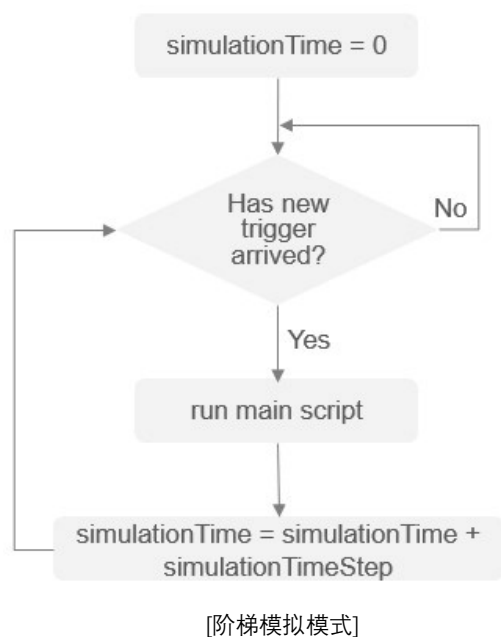
默认情况下，每个模拟周期由以下顺序操作组成：

1. 执行主脚本
2. 渲染场景

模拟

步进模式默认情况下，CoppeliaSim 通过**自动运行**模拟步骤来执行模拟。然而，在许多情况下，能够**手动触发**每个模拟步骤非常重要，例如，为了单独检查每个模拟步骤，或者为了将CoppeliaSim与外部应用程序同步，就是步进模式（或同步模式）

在 CoppeliaSim 中，可以通过sysCall_beforeMainScript回调函数轻松实现阶梯模拟。在外部应用程序中，通常有一个专用功能来启用阶梯模式，然后触发每个单独的步骤。下面代码说明了来自 Python ZeroMQ 远程 API 客户端的阶梯模式：



```
import time
from zmqRemoteApi import RemoteAPIClient

client = RemoteAPIClient()
sim = client.getObject('sim')

client.setStepping(True)

sim.startSimulation()
while (t := sim.getSimulationTime()) < 3:
    s = f'Simulation time: {t:.2f} [s]'
    print(s)
    client.step()
sim.stopSimulation()
```

模拟

模拟对话框可以使用[菜单栏-->模拟-->模拟设置]或单击以下工具栏按钮访问模拟对话框：



[模拟工具栏按钮]

Simulation Settings

Time steps

Simulation dt: 0.0500

Dynamics dt (effective dt=5.0ms): 0.0050

Simulation

Simulation passes per frame (ppf): 1

☐ Real-time simulation, multiplication factor: 1.00

☐ Pause when simulation time higher than: 10.00

☒ Pause on script error

☐ Full screen at simulation start

Dynamics

☒ Dynamics enabled, engine: Bullet 2.78

☐ Display contact points: Engine properties

Gravity vector: +0.00 +0.00 -9.81

At simulation end

☒ Reset scene to initial state

☒ Remove new objects

[模拟设置对话框]

模拟

Simulation dt: 模拟时间步长。执行主脚本时模拟时间会增加**模拟 dt**。建议默认模拟 dt保留为 50 毫秒。模拟dt不小于动态dt

Dynamics dt: 物理引擎使用的时间步长。使用较大时间步长会导致仿真速度更快但精度低。较小时间步长将导致更精确的模拟，但需更多时间。建议将**默认动态 dt**保持在 5 毫秒。结合 50ms 的默认仿真dt，每次仿真过程需要 10 个动力学计算步骤。

Simulation passes per frame : 一个渲染通道的**模拟通道数**。值为 10 表示在刷新屏幕之前执行主脚本 10 次（10 个模拟步骤）。

Real-time simulation,multiplication factor: 如果选择，则仿真时间将被**固定为实时**。乘法因子 X 会将仿真时间实时固定为 10*。

Pause when simulation time higher than: 允许指定模拟将暂停的**模拟时间**（例如，能够在特定的模拟时间分析某些结果）。

Pause on script error: 如启用，则当嵌入式脚本中发生**脚本错误**时，模拟将**暂停**。

Full screen at simulation start: 如启用则模拟以**全屏模式**启动。在全屏模式下对话框和消息不会显示，鼠标左键处于活动状态。仅当场景配置正确并最终确定后，才建议用该模式。全屏模式可使用 esc 键保留，在仿真期间通过bool 参数sim.booparam_fullscreen切换。Unler Linux 和 MacOS 全屏模式可能仅部分受支持，在某些系统上切换回正常模式可能会失败。

Dynamics enabled: 启用或禁用物理引擎。引擎允许选择一个**物理引擎**。每个引擎都有特定的优点和缺点。一个引擎运行良好的模拟并不总是意味着它也可以在其他引擎上运行良好。

Display contact points: 显示或隐藏物理引擎计算的**接触点**。

Engine properties: 每个引擎都有可以全局设置的**特定参数**。阅读相应的引擎文档，以实现最佳参数设置。

Gravity vector: 施加到所有非静态形状（例如重力）的恒定力的**振幅和方向**。

Reset scene to initial state: 选中后，所有对象的大部分参数和属性都将重置为**模拟开始**时的状态。

Remove new objects: 选中后，在模拟期间添加的场景对象将在**模拟结束**时删除。

镜像安装

1. 下载虚拟机

首先，打开VMware虚拟机官网：

<https://www.vmware.com/cn/products/workstation-pro.html>

点击下载，并按照其安装流程一步一步安装，没有难点。

下载好之后，桌面显示如下图标，双击即可运行：

镜像安装

2. 导入镜像

1.  仿真系统:

[Ubuntu 64位.ovf](#) ;

[Ubuntu 64位.mf](#) ;

[Ubuntu_64_位-disk1.zip](#) ;

[Ubuntu_64_位-disk1.z01](#) ;

[Ubuntu_64_位-disk1.z02](#) ;

[Ubuntu_64_位-disk1.z03](#) ;

[Ubuntu_64_位-disk1.z04](#)

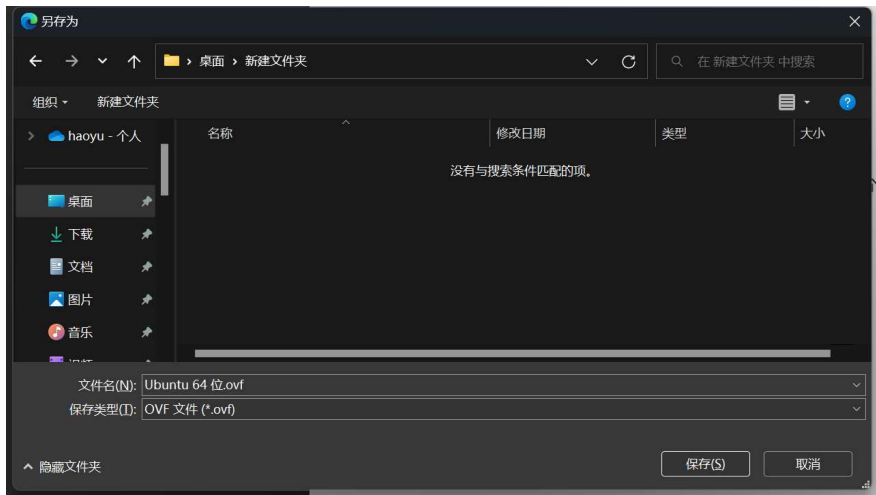
2.  操作指引:

[操作指引.pdf](#)

3.  仿真环境及代码样例

[仿真环境及代码样例.zip](#)

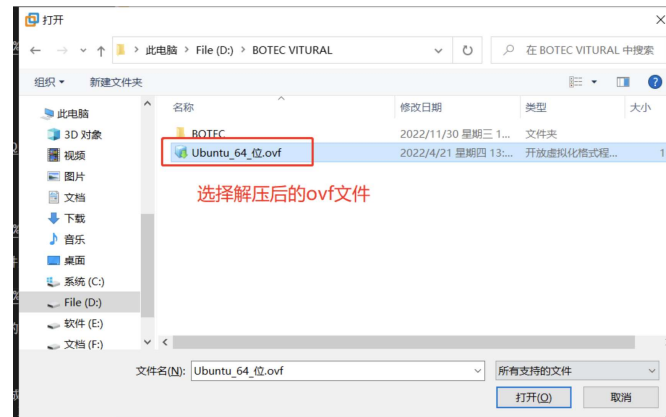
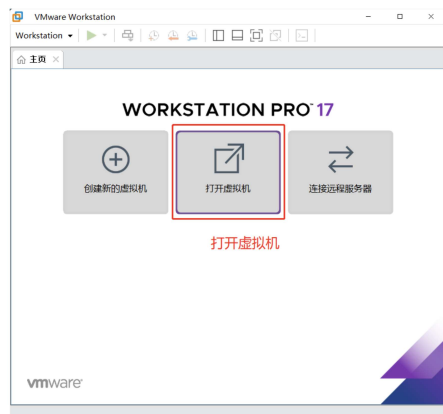
镜像安装



- Ubuntu 64 位.mf
- Ubuntu_64_位.ovf
- Ubuntu_64_位-disk1.vmdk

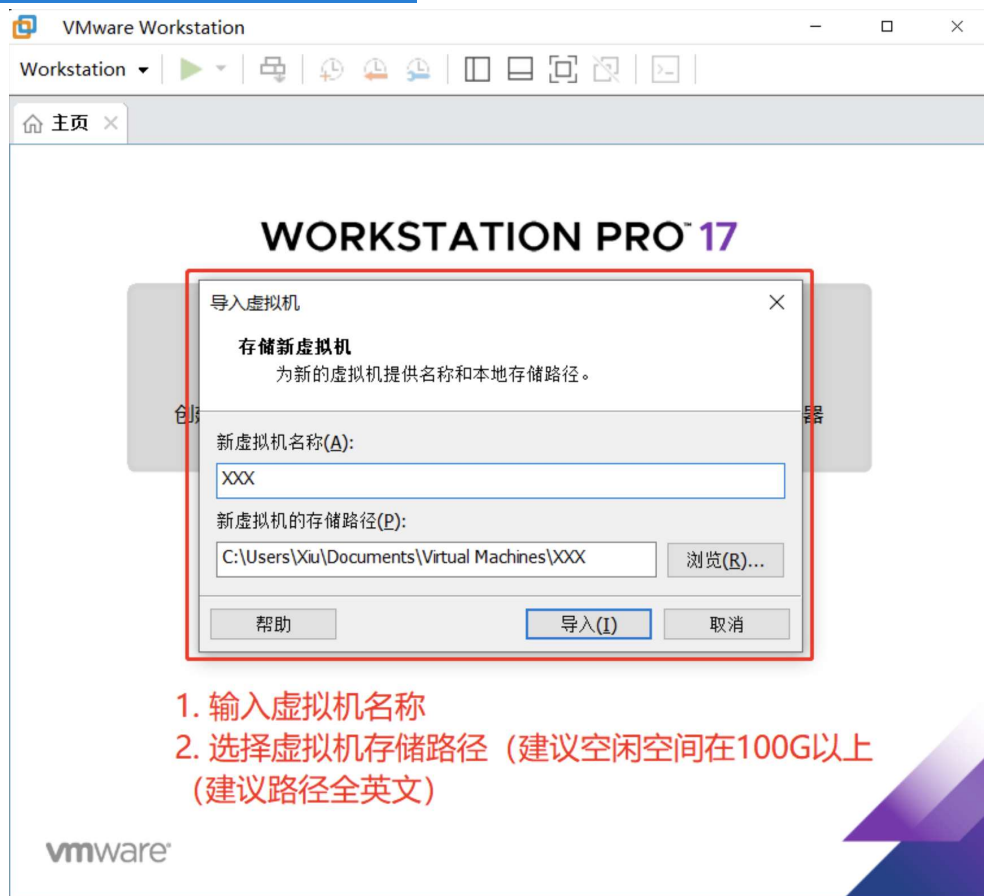
mf 文件、ovf 文件和 vmdk 文件应放在同一路径下。（建议路径为一个全英文路径）

镜像安装



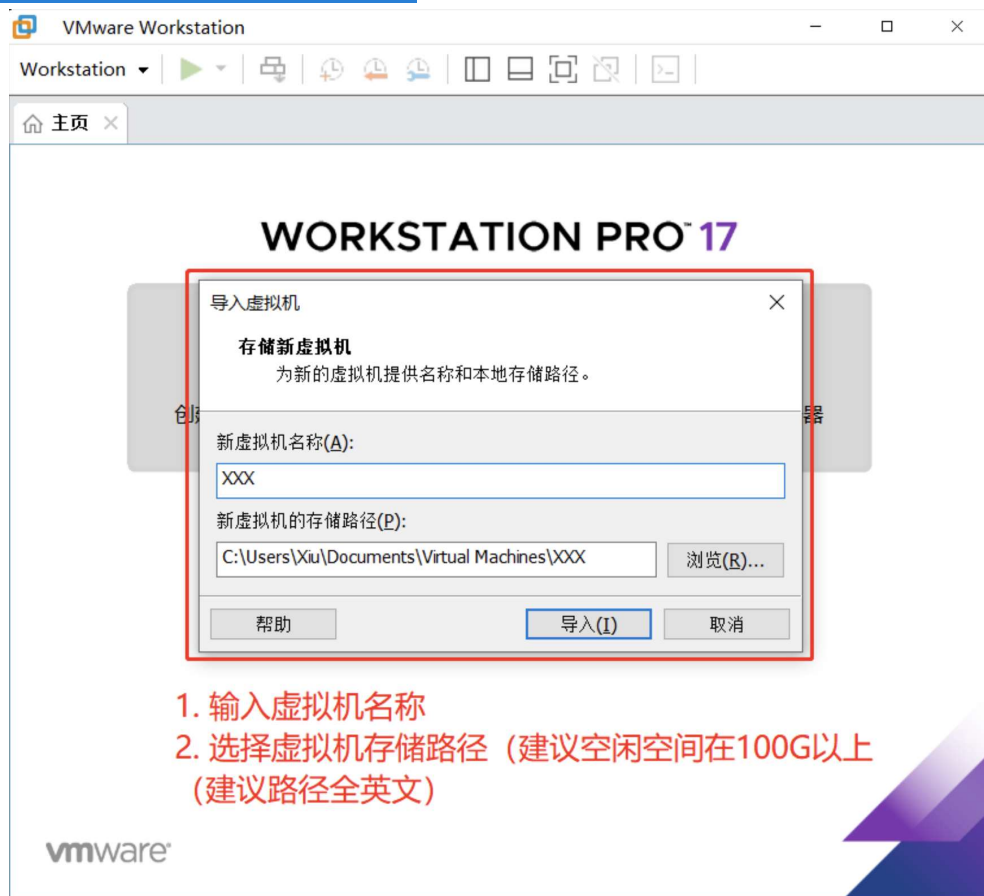
解压完成之后，打开 VMware 软件，选择 "打开虚拟机"，
导入刚才的 ovf 文件。

镜像安装



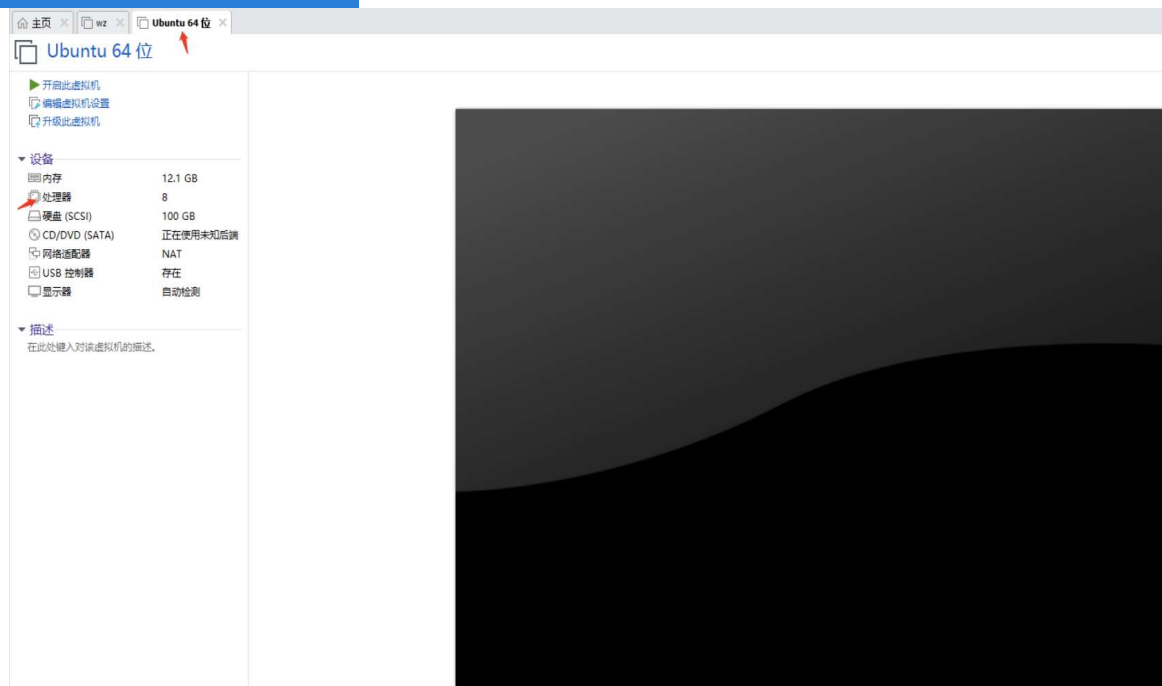
输入虚拟机名称，并选择虚拟机导入路径。
建议选择一个空闲空间在 100Gb 以上的全英文路径。

镜像安装



输入虚拟机名称，并选择虚拟机导入路径。
建议选择一个空闲空间在 100Gb 以上的全英文路径。

镜像安装



虚拟机用户名：fan

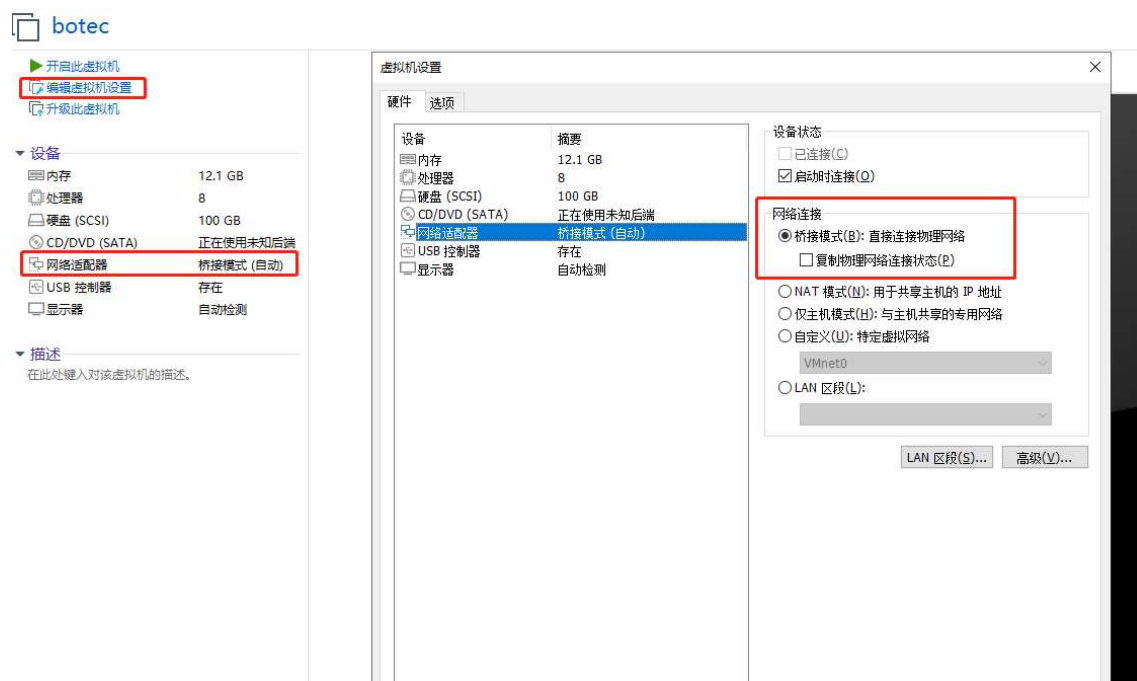
开机密码：121

若虚拟机Ubuntu系统开机时，VMware 提示：已将虚拟机配置成 16 个 CPU 运行，但主机只有 8 个CPU，此时关闭 ubuntu 虚拟机，并选择 ubuntu 64 位选项卡，双击设备下的处理器，并将 16 改为自己电脑的CPU核心数(建议仿真环境配置是 8 核 CPU + 8G 内存以上)。如下图所示：

镜像安装

3. 设置网络

使用“桥接模式”，能够连接外网，使用单独的 ip 地址，网段需要与桥接连接的外部网络的网段相同，可以点击右下角网络查看属性能得到 ip 信息。



点击编辑虚拟机设置，
选择硬件-网络适配器，选中桥接模式即可。

镜像安装

3. 更新程序

打开终端,输入更新指令:

```
sh -c "$(wget --no-cache http://roban.lejurobot.com/sys_update/download_sys_update_beta.sh -O -)"
```

如果弹出以下提示需要输入管理员密码用于配置用户组(后续使用sudo命令将不用输入密码):

```
启动升级工具...  
第一次更新需要输入管理员密码,初始密码请查看使用文档  
[sudo] password for fan:
```

则输入初始密码121 (此处输入密码不显示, 输入后按下回车即可)

镜像安装

```
当前机器人的软件版本: 0.2.0-79-gc59ece3
Roban 更新程序
当前程序更新脚本所在分支:
当前程序更新脚本版本号:
目标更新分支: master
    01. 更新 Roban 程序
    02. 更改配置文件
    03. 重新编译 Roban 程序
> 04. 强制更新 Roban 程序(会删除自定义文件)
    05. 更新依赖库
退出程序
```

第一次导入虚拟机后需要使用“04强制更新”一次,后续可以使用“01更新Roban程序”普通更新即可。



Webots仿真环境



Webots

OPEN SOURCE ROBOT SIMULATOR



stars 1.3k

downloads 359k



Download

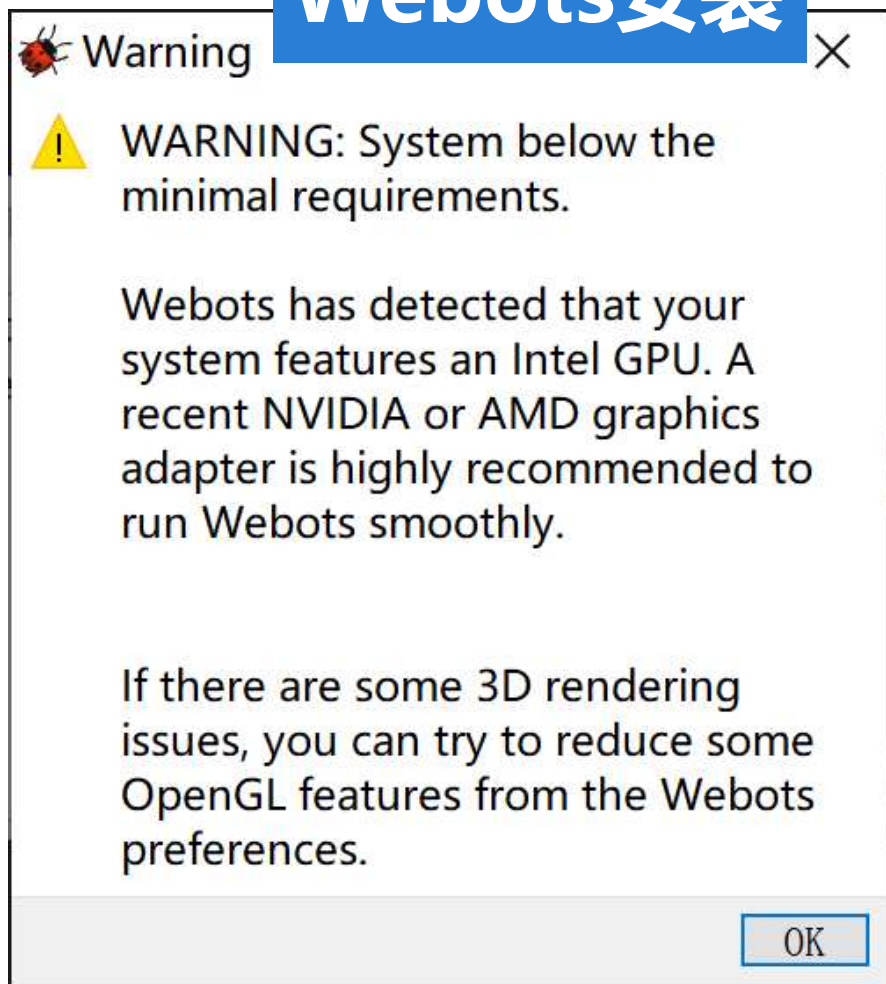


Windows - 1.63 GB

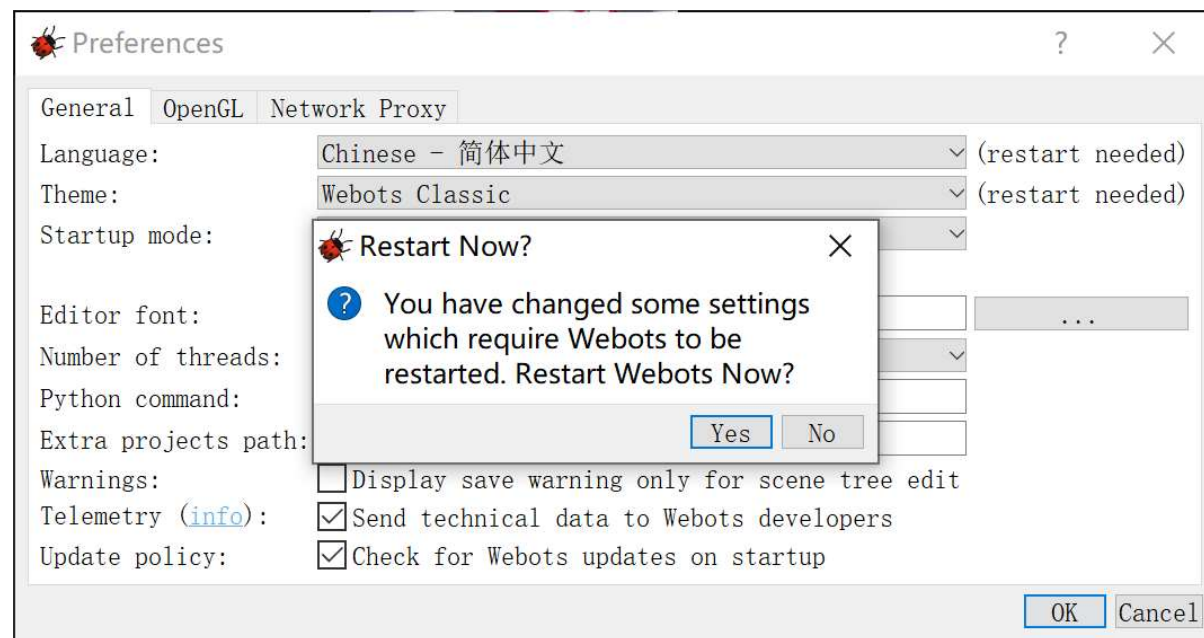
15 Dec 2020 - R2021a



Webots安装



3D渲染需要计算机硬件支持



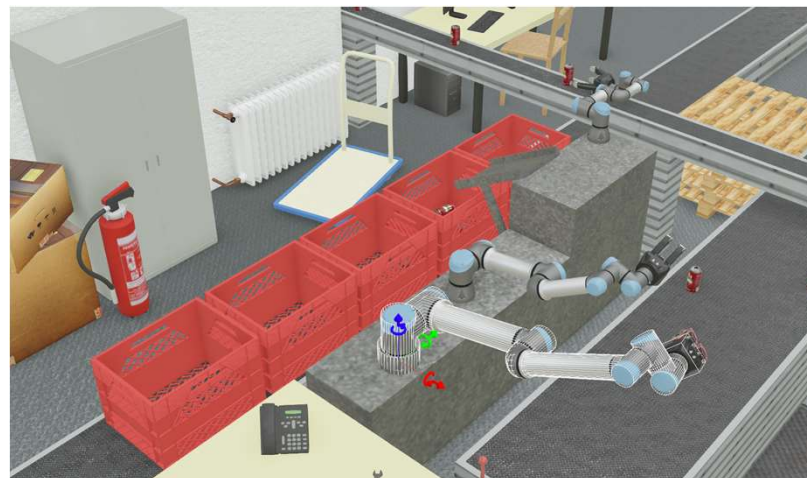
可以部分汉化

用户指导手册

- Help>User Guide
 - 内容包括 建模、编程、仿真、迁移
 - 有在线版和离线版



- Help>Webots Guided Tour
 - Webots 所含的预置机器人和场景19个
 - Devices Tour 包含设备的例子43个



用户指导手册



- File>Open Sample World
 - Webots提供大量的例子
 - 人、编程语言、机器人、交通工具
 - NAO机器人的例子，在机器人>Softbank>Nao下面。
 - Abb 机器人提供了逆动力学的例子，可以参考。安装python ikpy package
 - 有名的机器人
 - boston dynamic公司的atlas、spot
 - irobot扫地机器人



PR2



HOAP-2



Atlas



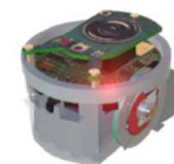
NAO



Mavic 2 Pro



irobot



epuck

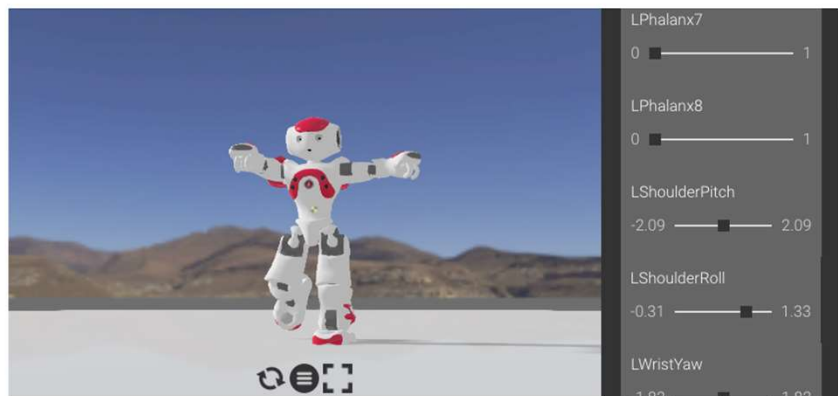
用户指导手册——NAO

- Webots User Guide>Robots>Nao 请仔细阅读
- NAO机器人的关节和控制效果（在线版才有）

Nao Models: 可以设置version为V5版本,
degreeOfFreedom为25自由度

Webots User Guide R2021a

SoftBank Robotics' Nao



The Nao robot is a humanoid robot developed by [SoftBank Robotics](#) (formerly known as Aldebaran Robotics). This section explains how to use Nao robot simulated in Webots. Currently, Webots supports the Nao v3.3, v4.0 and v5.0 versions, with and without their articulated fingers (respectively with 25 and 21 degrees of freedom) for the first two.



C:\Program Files\Webots\projects\robots\softbank\ nao\worlds\challenge.v...bots R2021a

文件(F) 编辑(E) 查看(V) 模拟(S) 生成(B) Overlays 工具(T) 向导(W) 帮助(H)

NAO Challenge

场景树

- > CeilingLight
- > NaoRoom
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF LANDMARK VisualArmature
 - > DEF PAINTING_BIG VisualArmature
 - > DEF PAINTING_TOP VisualArmature
 - > DEF PAINTING_MIDDLE VisualArmature
 - > DEF PAINTING_VERTICAL VisualArmature
 - > DEF CALENDAR VisualArmature
 - > DEF TABLE SolidBox
 - > Key
 - > KeyHole
 - > KeyPot
 - > Tray
 - > Distributor

Selection: controller (String)

nao_demo

选择... 编辑

Console - All

```
[B]: print foot bumpers
[Home][End]: print scaled top/bottom camera image
[PageUp][PageDown]: open/close hands
[7][8][9]: change all leds RGB color
[0]: turn all leds off
```

菜单

工具栏

拖拽上传

Text Editor

代码

设计视图

左键旋转，右键平移，中间滚轮放大缩小视图

控制台

用户指导手册——机器人运行时的相关信息

NAO相关信息：

Accelerometer 加速度计

Camera 摄像头

DistanceSensor 距离感知器

GPS 定位

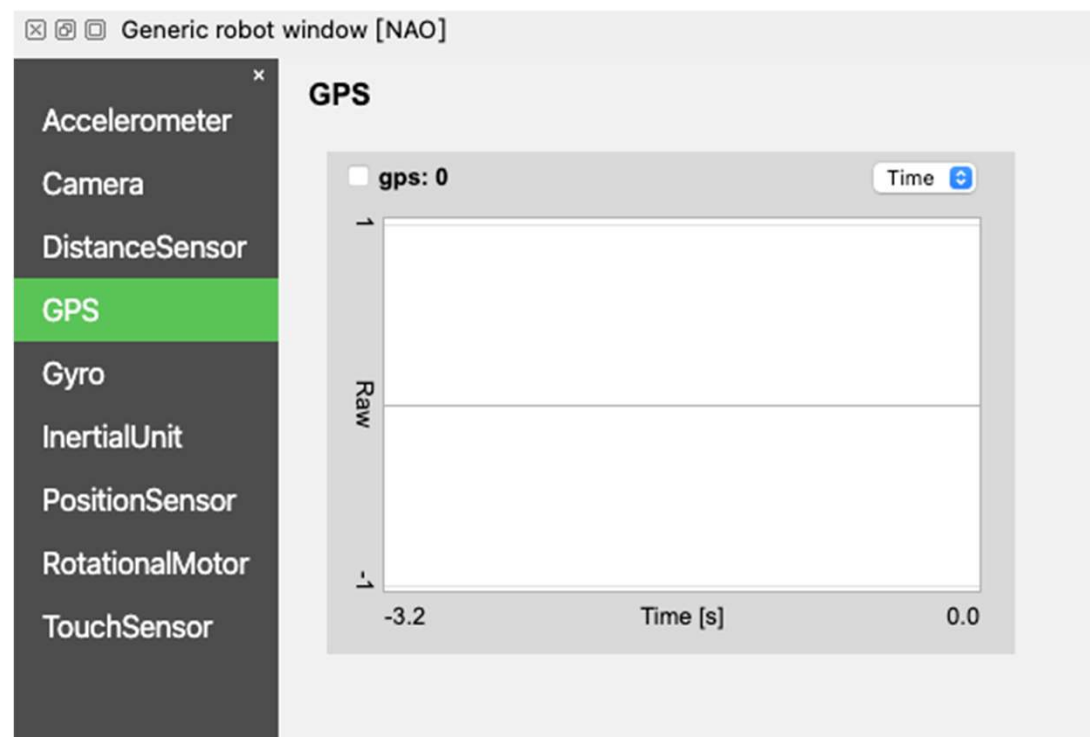
Gyro 陀螺仪：测量相对角速度

InitalUnit 惯性单元：测量滚动、俯仰和偏航角。

PositionSensor 监控关节的位置

RotationalMotor：关节的旋转角度

TouchSensor 触碰传感器



机器人node上，右键菜单，选择“Show Robot Window”

Webots简介—构成

- Webots搭建世界 (world)
 - 使用Wizard 新建项目 (project) 比新建world更便利
 - 节点 (node) 与节点属性 (物品、机器人、光线、背景)
 - 节点基本使用VRML97语法描述 (本课程无需学习VRML语法)
 - 基础节点和PROTO节点, 可使用关键词检索
 - PROTO节点是在文本文件.proto定义的, 更多请参考reference/proto
 - 对于旋转rotation使用axis-angle表示, 前三个参数为单位轴, 第四个参数为旋转角
- 控制器, 编写程序控制机器人
 - 参考User Guide / Controller Programming
 - 注意文件路径
 - 亦可以使用外部控制器